



Oracle Database 23 AI New Features

An Accenture White Paper by

Kumar Bhashkar Alok

(kumar.bhashkar.alok@accenture.com)

February 2026



Table of Contents:

Oracle Database 23 AI New Features	1
1.EDITOR'S	3
2.REVIEWED BY.....	3
3.OBJECTIVE	4
4.DISCLAIMER.....	4
5.OVERVIEW.....	4
6. Oracle 23ai New Features	5
6.1 AI Vector Search.....	5
6.2 JSON Relational Duality.....	8
6.3 SQL Enhancements.....	10
6.4 Performance Features.....	20
7. CONCLUSION	22
8.REFERENCES	22



1.EDITOR'S

S.No	Date	Author	Email Id	Project Role
1	02-Feb-2026	Kumar Bhashkar Alok	kumar.bhashkar.alok@accenture.com	Assoc. Manager

2.REVIEWED BY

Name	Author	Project/Org Role	Review Date



3.OBJECTIVE

The objective of this white paper is to provide an in-depth understanding of the Oracle 23 AI new features.

4.DISCLAIMER

The information provided in this white paper is based on the knowledge available at the time of writing, up to February 2026. While every effort has been made to ensure the accuracy of the content and there may be updates or changes that are not reflected in this document. Readers are advised to refer to the official documentation and community resources for the most up-to-date information.

5.OVERVIEW

Oracle 23ai New Features:

Oracle 23ai delivers built-in AI capabilities, modern data models, and performance enhancements to support intelligent, scalable data-driven applications. AI Vector Search enables similarity-based searches using vector embeddings to retrieve data by semantic meaning directly within the database. JSON Relational Duality allows data to be simultaneously represented as JSON documents and relational tables without duplication or inconsistency. SQL Enhancements extends SQL with richer functionality to simplify analytics, AI integration, and JSON processing. Performance Features introduces optimizations that improve query speed, scalability, and efficiency across all workload types.

6. Oracle 23ai New Features

6.1 AI Vector Search

Oracle AI Vector Search is a powerful feature introduced in Oracle Database 23ai that allows you to perform similarity searches on vectors. This feature enables you to search for vectors that are like a query vector, which are useful for applications like recommendation systems, semantic search, and natural language processing.

AI Vector Search allows you to search data based on meaning (semantic similarity) instead of exact words or values.

- 1) Traditional search: Matches keywords or exact values
- 2) AI Vector Search: Matches intent, context, and meaning

Oracle Database 23ai stores vector embeddings directly in the database and performs similarity search efficiently.

Traditional search problem:

if you search: “cheap mobile phone”

Traditional search would be something like this:

```
WHERE product_description LIKE '%cheap%'
```

Where it is missed below words where words are different, even though meaning is same.

- budget smartphone
- low-cost phone
- affordable mobile

AI Vector Search Solution:

Step 1: AI converts text into vectors (numbers) that represent meaning. A vector is an array of numbers that represent meaning. Example

```
cheap mobile phone    → [0.21, 0.87, 0.44, ...]
budget smartphone     → [0.22, 0.86, 0.45, ...]
affordable mobile     → [0.20, 0.88, 0.43, ...]
```

These vectors are numerically close because the meaning is similar.

Step 2: Similarity search

Oracle compares vectors using **distance calculations** (like cosine similarity).

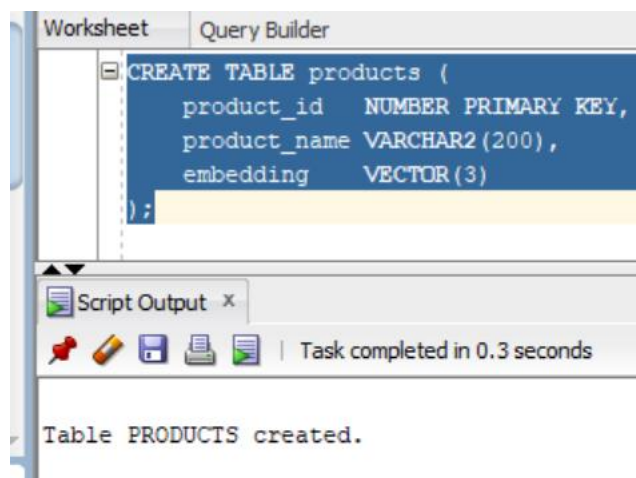
- Small distance = similar meaning
- Large distance = different meaning

So, the search for cheap phone matches budget smartphone and affordable mobile.

Even though words are different, intent is same. That's why AI Vector Search solves the synonym and intent problem that traditional search cannot.

Example:

Step 1: Create table.

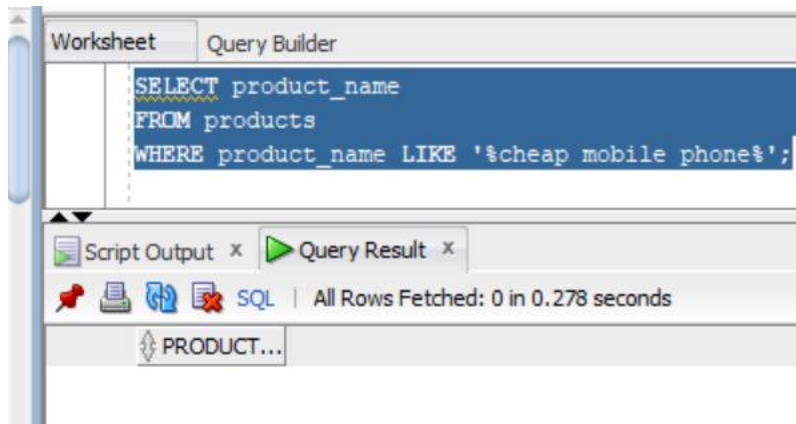


Step 2: insert sample data

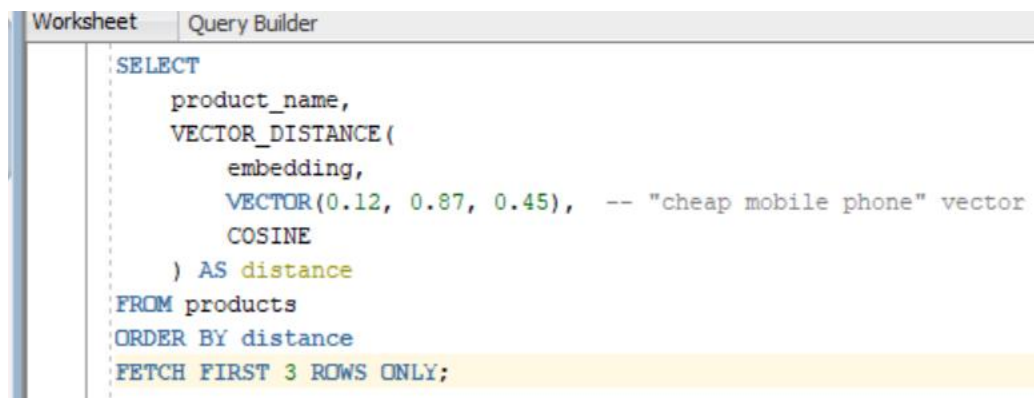
```

INSERT INTO products VALUES ( 1, 'budget smartphone', VECTOR(0.12, 0.88, 0.45));
INSERT INTO products VALUES ( 2, 'low-cost phone', VECTOR(0.13, 0.87, 0.44));
INSERT INTO products VALUES ( 3, 'affordable mobile', VECTOR(0.11, 0.89, 0.46));
INSERT INTO products VALUES ( 4, 'premium flagship smartphone', VECTOR(0.90, 0.10, 0.05));
COMMIT;
  
```

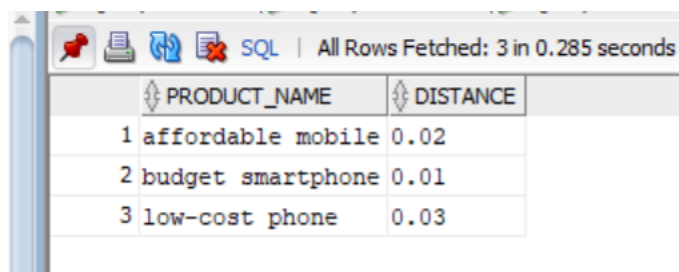
Step 3: Traditional search doesn't give result



Step 4: Vector search(Semantic search)



O/P - gives all results basis meaning(shorter vector distance)



SQL | All Rows Fetched: 3 in 0.285 seconds

	PRODUCT_NAME	DISTANCE
1	affordable mobile	0.02
2	budget smartphone	0.01
3	low-cost phone	0.03

Benefits:

- a) Vectors can be stored directly on relational tables
- b) Similar meaning → shorter vector distance
- c) No external vector DB required

d) Fully Oracle-compliant SQL

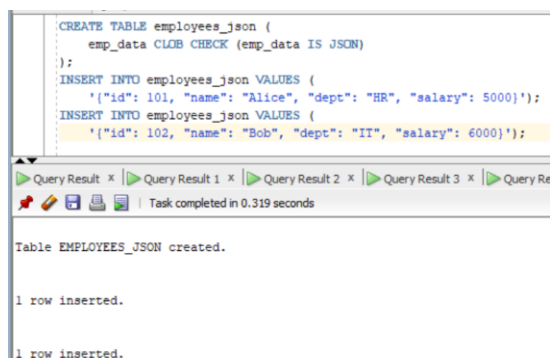
6.2 JSON Relational Duality

JSON-Relational Duality in Oracle means the same data can be treated in two ways: as a JSON document (like in modern apps or APIs) or as relational rows and columns (like in traditional databases). Data can be stored as JSON in a table and then can be queried directly as structured columns without duplicating data. This allows developers to work with flexible JSON formats while still leveraging SQL analytics and reporting. It makes applications simpler because you don't need two separate data models — one table can serve both document-style and relational-style queries. In short, Oracle treats JSON and relational formats as two sides of the same coin.

Before 23ai, querying JSON data was more cumbersome because Oracle did not allow treating JSON like relational columns directly in SELECT without using functions.

- JSON_VALUE for scalar values
- JSON_TABLE for relational projection

Example: create table employees_json with a clob column as emp_data



```
CREATE TABLE employees_json (  
  emp_data CLOB CHECK (emp_data IS JSON)  
);  
INSERT INTO employees_json VALUES (  
  '{"id": 101, "name": "Alice", "dept": "HR", "salary": 5000}');  
INSERT INTO employees_json VALUES (  
  '{"id": 102, "name": "Bob", "dept": "IT", "salary": 6000}');
```

Query Result x | Query Result 1 x | Query Result 2 x | Query Result 3 x | Query Re

Task completed in 0.319 seconds

Table EMPLOYEES_JSON created.

1 row inserted.

1 row inserted.

Before 23ai, data in the table were assessed using JSON_VALUE like below

Worksheet Query Builder

```
SELECT
  JSON_VALUE(emp_data, '$.id') AS emp_id,
  JSON_VALUE(emp_data, '$.name') AS emp_name,
  JSON_VALUE(emp_data, '$.dept') AS department,
  JSON_VALUE(emp_data, '$.salary') AS emp_salary
FROM employees_json;
```

Query Result x

SQL | All Rows Fetched: 2 in 0.328 seconds

	EMP_ID	EMP_NAME	DEPARTMENT	EMP_SALARY
1	101	Alice	HR	5000
2	102	Bob	IT	6000

Or using JSON_TABLE

```
SELECT
  FROM employees_json e,
  JSON_TABLE(e.emp_data, '$'
    COLUMNS (
      id          NUMBER PATH '$.id',
      name        VARCHAR2(50) PATH '$.name',
      dept        VARCHAR2(50) PATH '$.dept',
      salary      NUMBER PATH '$.salary'
    )
  ) jt;
```

Query Result x

SQL | All Rows Fetched: 2 in 0.649 seconds

EMP_DATA	ID	NAME	DEPT	SALARY
1 {"id": 101, "name": "Alice", "dept": "HR", "salary": 5000}	101	Alice	HR	5000
2 {"id": 102, "name": "Bob", "dept": "IT", "salary": 6000}	102	Bob	IT	6000

Now with 23 ai, advancements. We can have the same JSON data but now treated naturally as relational columns. Simplifies analytics and reporting.

Dual usage: API-ready JSON + relational SQL in one table

```

SELECT
  emp_data.id AS emp_id,
  emp_data.name AS emp_name,
  emp_data.salary AS emp_salary
FROM employees_json,
     JSON_TABLE(emp_data, '$'
        COLUMNS (
            id      NUMBER PATH '$.id',
            name    VARCHAR2(50) PATH '$.name',
            dept    VARCHAR2(50) PATH '$.dept',
            salary  NUMBER PATH '$.salary'
        )
     ) emp_data;

```

Query Result x

SQL | All Rows Fetched: 2 in 0.329 seconds

	EMP_ID	EMP_NAME	EMP_SALARY
1	101	Alice	5000
2	102	Bob	6000

6.3 SQL Enhancements

6.3.1 SELECT without FROM

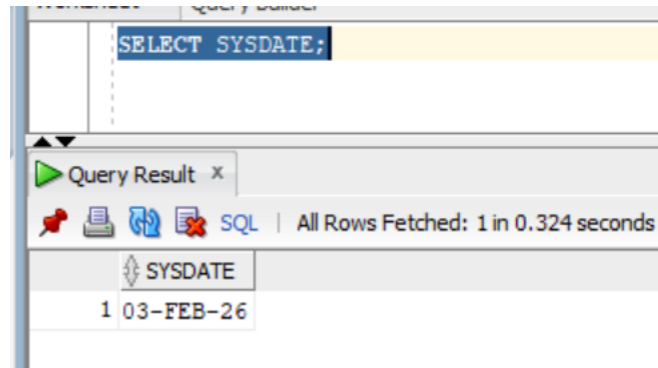
In Oracle Database 23ai, Oracle removed the requirement to select from the dummy DUAL table. You can now run `SELECT <expression>;` directly for system, constant, or session-based functions like `SYSDATE`. This simplifies SQL syntax and aligns Oracle with modern SQL standards.

```

SELECT SYSDATE;
SELECT SYSTIMESTAMP;
SELECT MOD(10, 3);
SELECT SQRT(16);
SELECT LOWER('ORACLE');
SELECT UPPER('oracle');

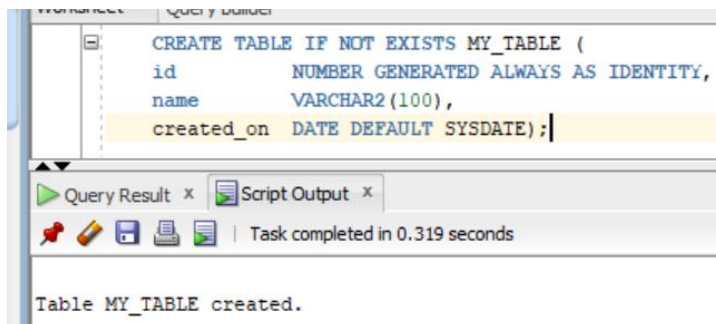
```

Example:

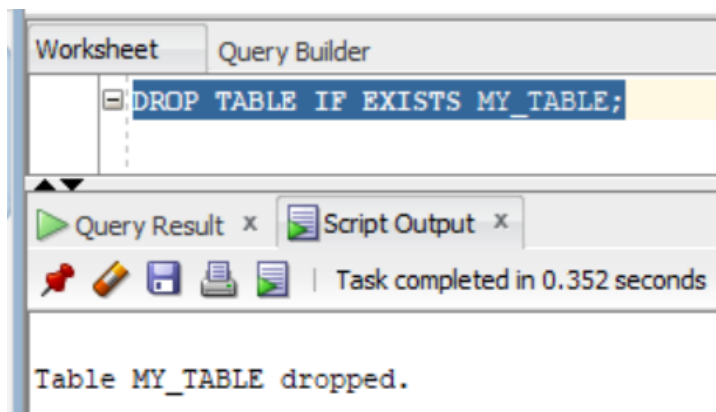


6.3.2 IF [NOT] EXISTS modifiers in DDL

- a) Creates table only when table is not already existing



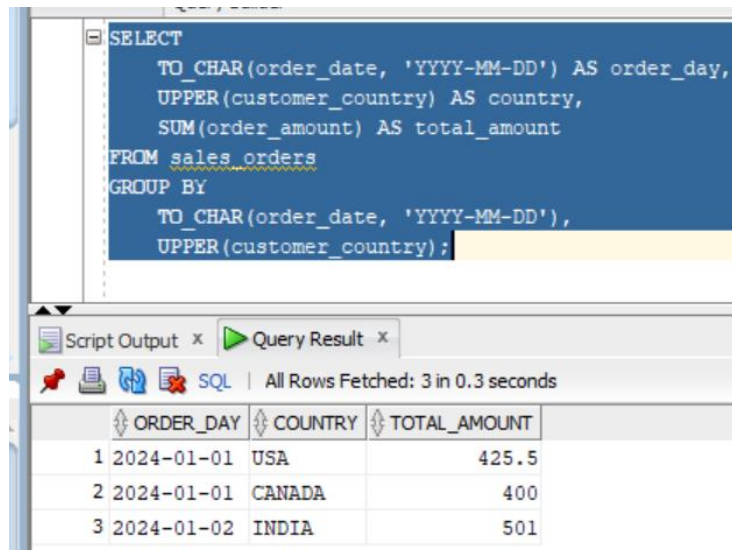
- b) Drops are executed only when table exists



6.3.3 GROUP BY with column alias/position

In Oracle Database 23ai, GROUP BY now supports column aliases and select-list positions, making aggregation queries easier to read and maintain. This reduces duplication of complex expressions in the GROUP BY clause.

Early version: whole expression in group by clause



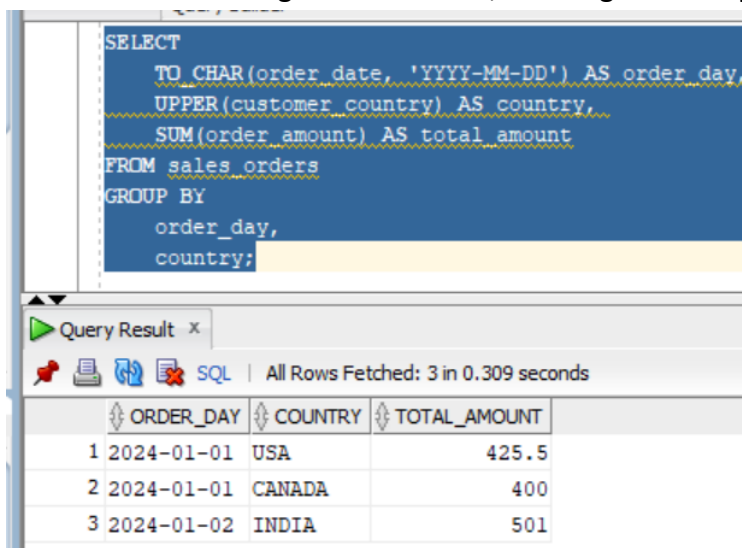
```

SELECT
  TO_CHAR(order_date, 'YYYY-MM-DD') AS order_day,
  UPPER(customer_country) AS country,
  SUM(order_amount) AS total_amount
FROM sales_orders
GROUP BY
  TO_CHAR(order_date, 'YYYY-MM-DD'),
  UPPER(customer_country);

```

	ORDER_DAY	COUNTRY	TOTAL_AMOUNT
1	2024-01-01	USA	425.5
2	2024-01-01	CANADA	400
3	2024-01-02	INDIA	501

In 23 ai version: Using column aliases, avoiding whole expression



```

SELECT
  TO_CHAR(order_date, 'YYYY-MM-DD') AS order_day,
  UPPER(customer_country) AS country,
  SUM(order_amount) AS total_amount
FROM sales_orders
GROUP BY
  order_day,
  country;

```

	ORDER_DAY	COUNTRY	TOTAL_AMOUNT
1	2024-01-01	USA	425.5
2	2024-01-01	CANADA	400
3	2024-01-02	INDIA	501

6.3.4 Return old + new values with RETURNING INTO

In Oracle Database 23ai, the RETURNING INTO clause was enhanced so that for UPDATE and MERGE statements you can get both the before-update (OLD) and after-update (NEW) values directly from the DML statement itself.

This means:

UPDATE can return old and new values of the affected columns

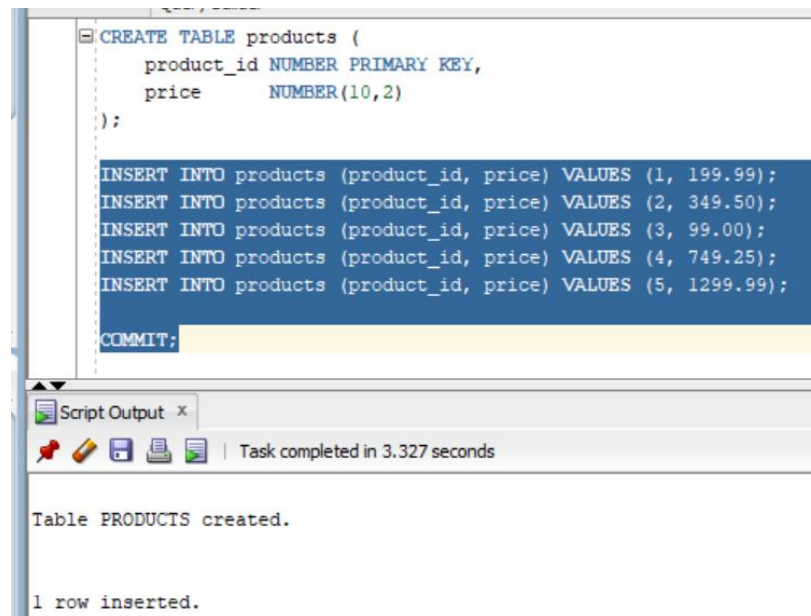
INSERT can return only new values

DELETE can return only old values

MERGE can return old and/or new values depending on action

This makes writing audit logic, change tracking, or application feedback much easier—without doing a separate SELECT before/after the DML.

Step 1: Create table



```

CREATE TABLE products (
  product_id NUMBER PRIMARY KEY,
  price      NUMBER(10,2)
);

INSERT INTO products (product_id, price) VALUES (1, 199.99);
INSERT INTO products (product_id, price) VALUES (2, 349.50);
INSERT INTO products (product_id, price) VALUES (3, 99.00);
INSERT INTO products (product_id, price) VALUES (4, 749.25);
INSERT INTO products (product_id, price) VALUES (5, 1299.99);

COMMIT;
  
```

Script Output x

Task completed in 3.327 seconds

Table PRODUCTS created.

1 row inserted.

Step 2: Output gives old and new price.

```

DECLARE
    v_old_price products.price%TYPE;
    v_new_price products.price%TYPE;
BEGIN
    UPDATE products
    SET price = price * 1.10
    WHERE product_id = 1
    RETURNING
        OLD price,
        NEW price
    INTO
        v_old_price,
        v_new_price;

    DBMS_OUTPUT.PUT_LINE('Old price: ' || v_old_price);
    DBMS_OUTPUT.PUT_LINE('New price: ' || v_new_price);
END;

```

Script Output x

Task completed in 0.574 seconds

Old price: 199.99
New price: 219.99

PL/SQL procedure successfully completed.

6.3.5 SQL Macros

SQL Macros are database objects that return a SQL fragment, not data. SQL Macros allow developers to write reusable, parameterized SQL logic that expands inline at parse time, combining the flexibility of functions with the performance of plain SQL.

A. Simple Scalar SQL Macro - Reuse salary increment logic.

Step 1: Create Macro

Worksheet Query Builder

```

CREATE FUNCTION add_bonus(p_salary NUMBER)
RETURN VARCHAR2
SQL_MACRO (SCALAR)
AS
BEGIN
    RETURN 'p_salary + (p_salary * 0.10)';
END;
/

```

Script Output x Query Result x

Task completed in 1.305 seconds

Function ADD_BONUS compiled

Step 2: Use Macro in SQL

```

SELECT emp_id,
       salary,
       add_bonus(salary) AS salary_with_bonus
FROM employees;

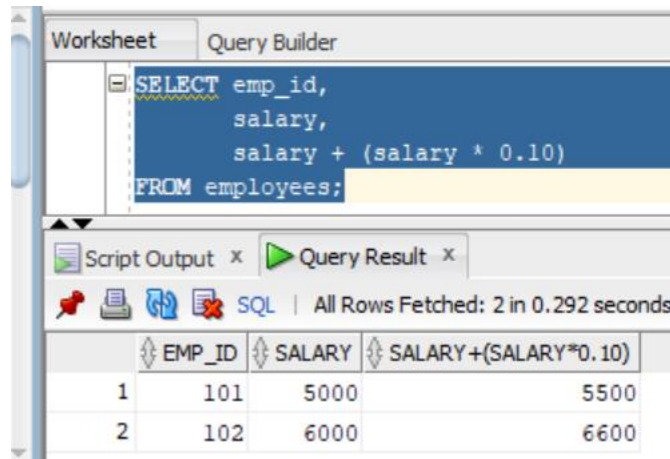
```

Script Output x Query Result x

All Rows Fetched: 2 in 0.289 seconds

EMP_ID	SALARY	SALARY_WITH_BONUS
1	101	5500
2	102	6600

Step 3: Oracle expands it internally to:



Worksheet Query Builder

```
SELECT emp_id,
       salary,
       salary + (salary * 0.10)
FROM employees;
```

Script Output x Query Result x

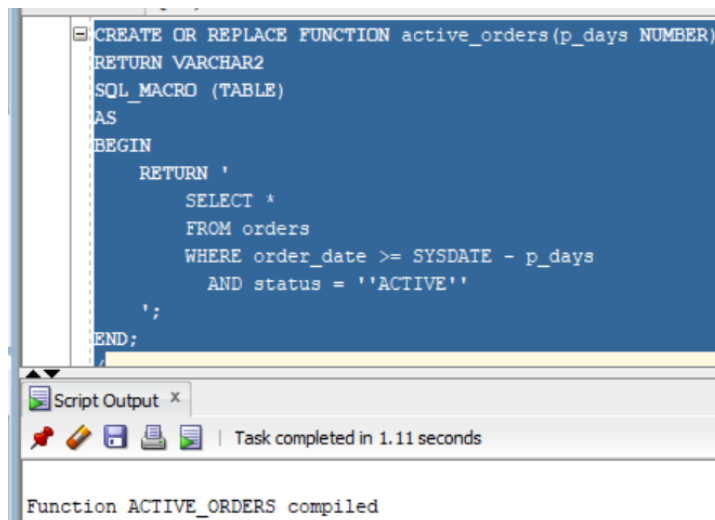
SQL | All Rows Fetched: 2 in 0.292 seconds

	EMP_ID	SALARY	SALARY+(SALARY*0.10)
1	101	5000	5500
2	102	6000	6600

Benefits: so, no function call at runtime and no performance penalty

B. Standardize filtering logic across queries.

Step 1: Create Macro



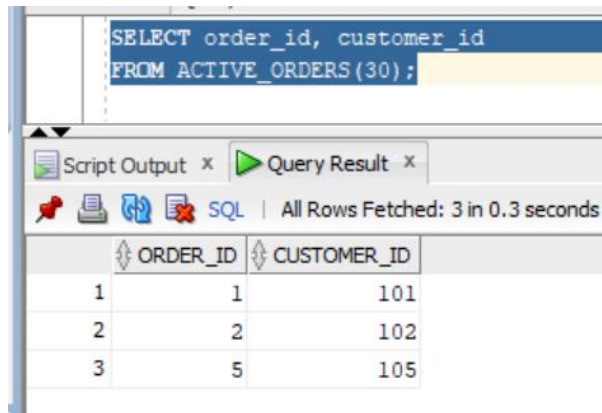
```
CREATE OR REPLACE FUNCTION active_orders(p_days NUMBER)
RETURN VARCHAR2
SQL_MACRO (TABLE)
AS
BEGIN
    RETURN '
        SELECT *
        FROM orders
        WHERE order_date >= SYSDATE - p_days
        AND status = ''ACTIVE''
    ';
END;
```

Script Output x

Task completed in 1.11 seconds

Function ACTIVE_ORDERS compiled

Step 2: Use Macro Like a Table



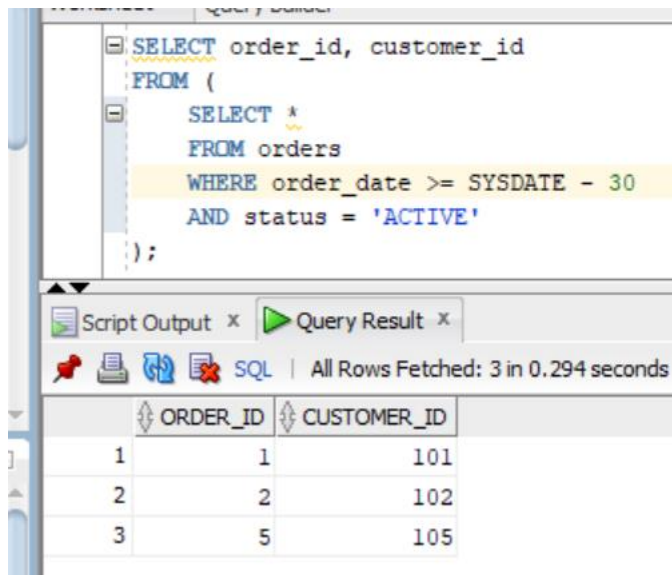
SELECT order_id, customer_id
FROM ACTIVE_ORDERS(30);

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.3 seconds

	ORDER_ID	CUSTOMER_ID
1	1	101
2	2	102
3	5	105

Step 3: Expanded SQL (Internally)



```
SELECT order_id, customer_id
FROM (
  SELECT *
  FROM orders
  WHERE order_date >= SYSDATE - 30
  AND status = 'ACTIVE'
);
```

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.294 seconds

	ORDER_ID	CUSTOMER_ID
1	1	101
2	2	102
3	5	105

Benefits: Looks like a table, behaves like a view but parameterized

C. Replacing Complex CASE Logic

Avoid repeating CASE expressions.

Step 1: Create macro

```
CREATE FUNCTION order_priority(p_amount NUMBER)
RETURN VARCHAR2
SQL_MACRO (SCALAR)
AS
BEGIN
    RETURN '
        CASE
            WHEN p_amount > 10000 THEN 'HIGH'
            WHEN p_amount > 5000 THEN 'MEDIUM'
            ELSE 'LOW'
        END
    ';
END;
```

Script Output x
Task completed in 1.099 seconds

Function ORDER_PRIORITY compiled

Step 2: call the function to void case. Performance is normal.

```
SELECT order_id,
       amount,
       order_priority(1000) AS priority
FROM orders;
```

Script Output x Query Result x
All Rows Fetched: 5 in 0.278 seconds

	ORDER_ID	AMOUNT	PRIORITY
1	1	5000	LOW
2	2	5000	LOW
3	3	5000	LOW
4	4	5000	LOW
5	5	5000	LOW

Benefits: case statement can be reused in the script multiple times without repeating them.

6.3.6 JavaScript Stored Procedures

Oracle 23ai lets you write and run stored procedures in JavaScript inside the database, just like PL/SQL.

Oracle embeds a JavaScript engine in the database, allowing you to:

- a) Write stored procedures in JavaScript
- b) Call them from SQL / PL/SQL
- c) Use them for business logic, data processing, APIs, AI pipelines

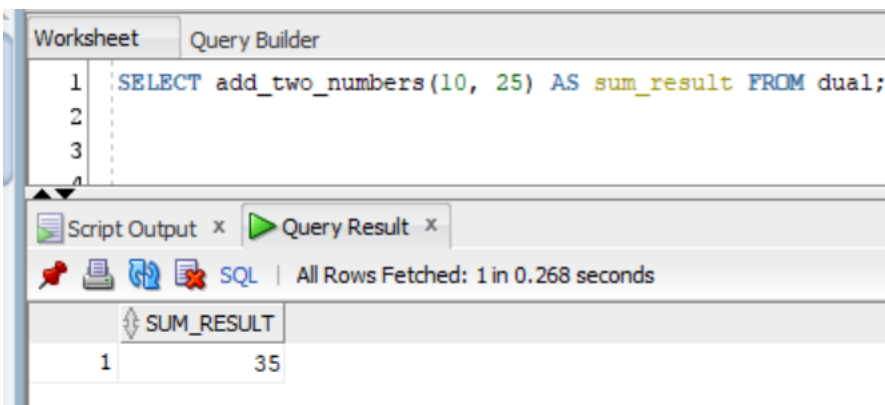
Step 1: Create function with JS

```

1 CREATE FUNCTION add_two_numbers(
2     num1 NUMBER,
3     num2 NUMBER
4 ) RETURN NUMBER
5 AS m1e LANGUAGE javascript
6 {{
7     // JavaScript logic: add the two numbers
8     return num1 + num2;
9 }};
10 /
11

```

Step 2: Call function



The screenshot shows the Oracle SQL Developer interface. The 'Query Builder' tab is active, displaying the following SQL query:

```
SELECT add_two_numbers(10, 25) AS sum_result FROM dual;
```

Below the query editor, the 'Query Result' tab is open, showing the execution results. The status bar indicates 'All Rows Fetched: 1 in 0.268 seconds'. The result is displayed in a table with one column, 'SUM_RESULT', and one row containing the value 35.

SUM_RESULT
35

How it helps:

Normally:

App → Java/Python/Node → DB

With JS stored procedure:

App → Oracle DB (JS logic inside)

Benefits:

- Faster performance
- Less application code
- Easier maintenance
- Reduces middle layer code

6.4 Performance Features

6.4.1 AI-Driven Automatic Indexing & Optimization

Oracle 23ai uses machine-learning–based automation to manage indexes and performance tuning without constant DBA intervention.

How it works:

The database continuously: Monitors real query workloads. Learns query patterns (filters, joins, access paths)

Automatically:

- a) Creates indexes when they help
- b) Drops indexes when they hurt or are unused
- c) Adjusts index types and usage over time
- d) This is done using AI models embedded in the database engine, not external tools.

Benefits:

- a) Eliminates guesswork in index creation

- b) Prevents over-indexing (which slows writes)
- c) Adapts as workloads change (OLTP vs analytics vs AI queries)
- d) Improves performance without manual tuning cycles

6.4.2 In-Memory Enhancements with Intelligent Workload Analysis.

Oracle Database In-Memory in 23ai becomes smarter and more autonomous, especially for mixed workloads (transactions + analytics + AI).

How it works: The database analyzes query frequency, data access patterns, and joins. Automatically decides which tables or columns should live in memory, how much memory to allocate and when to evict or retain objects.

Optimizes:

- a) Joins
- b) Aggregations
- c) Scans (columnar, vectorized execution)

Benefits:

- a) Faster analytics and reporting (often 10x–100x)
- b) Reduced CPU usage
- c) Lower latency for AI-driven queries that scan large datasets
- d) No manual memory tuning required

6.4.3 True Cache (Always-Consistent In-Memory Cache)

True Cache is a distributed, in-memory cache that is:

- a) Read-only
- b) Always transactionally consistent
- c) Automatically synchronized with the primary database

Think of it as a live, intelligent extension of the database, not a stale cache.

Benefits:

- a) Frequently accessed data is cached in memory
- b) Changes in the primary database are **instantly reflected**
- c) Applications read from cache transparently



- d) No application rewrites required
- e) Dramatically reduces database load

7. CONCLUSION

This paper highlights the understanding of Oracle Database 23ai new features, which position the database at the forefront of AI-driven and modern data management. By integrating capabilities such as AI Vector Search, JSON Relational Duality, enhanced SQL, and performance optimizations, Oracle 23ai provides an efficient and scalable foundation for unlocking data-driven insights. As data volumes and complexity continue to increase, Oracle Database 23ai will play a critical role in enabling intelligent, high-performance, and future-ready applications.

8. REFERENCES

- [What's New for Oracle AI Vector Search](#)
- <https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector-distance-functions-and-operators.html>
- [Document-Centric Use Case for JSON-Relational Duality](#)

